Cart-Pole Single Inverted Pendulum

Takeru Endo

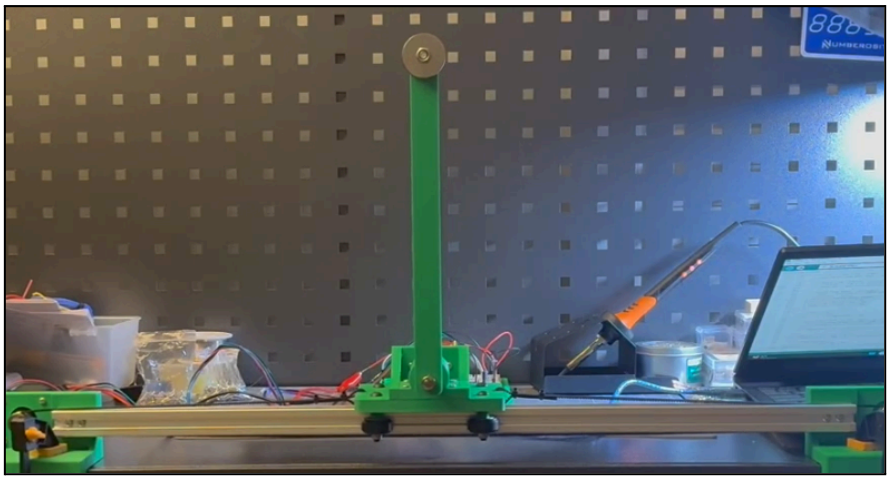
June 2026

1 Introduction

1.1 Overview

Designed, simulated, and fabricated a nonlinear single inverted pendulum, derived the equations of motion, implemented real-time PD control with state estimation and sensor filtering, and integrated embedded hardware for closed-loop stabilization.

1.2 Final Demonstration



From the demo video:
https://drive.google.com/file/d/1Ql43lMNHcwXh4a7Qc4OHIrRiB7-3618u/view?usp=sharing

Settling Time	- Fully settles (within ±0.5°) in ≈ 0.7s - Cart re-centres within ±1 cm in ≈ 1.4s
Overshoot	- Pendulum swings through upright to the opposite side by ≈ 60–65% of θ_0. - Cart peak excursion ≈ 2.5cm to 4.7cm. - Stays inside the 21.4 cm rail half-travel. - Settles with little to no sustained oscillation.
Maximum Recoverable Angle	-7°~7°
Sampling Frequency	200Hz

2 System Modeling

2.1 Summary

The cart-pole system was modeled as a nonlinear, coupled mechanical system using Lagrange's method. The model includes the cart's horizontal external force (Newtons) and the pendulum's external rotational moment (Nm). Then, the equations were linearized using small angle assumptions along the upright equilibrium position of the pendulum, leading to the state-space model.

2.2 Lagrange's Method

The following assumptions were made when deriving the equations of motion: the pendulum rod is rigid, the pendulum motion is confined to a single vertical plane, the cart motion is confined to a single horizontal axis, wheel slip is neglected, air resistance is negligible, gravitational acceleration is constant, and all frictional effects are neglected. To account for the distributed mass of the pendulum assembly, the combined center of mass of the end mass (m_1) and pendulum rod (m_2), with its length defined as l, was determined rather than assuming a point mass at the pendulum tip. The pivot point of the pendulum on the cart (M) was defined as (x,0), where x represents the horizontal position of the cart. The location of the pendulum's center of mass was then determined. Position of the pendulum's centre of mass relative to the pivot (x, 0):

$$x_{cm} = x + \frac{lsin(\theta)(m_1 + \frac{m_2}{2})}{m_1 + m_2}$$

$$y_{cm} = \frac{lcos(\theta)(m_1 + \frac{m_2}{2})}{m_1 + m_2}$$

Total kinetic energy T (cart translation + pendulum translation and rotation about its own centre of mass):

$$T = \frac{M}{2}\cdot(x')^2 + \frac{m_1}{2}\cdot(x'^2 + 2l\cdot cos(\theta)\cdot x'\cdot\theta' + l^2\cdot(\theta')^2)$$
$$+ \frac{m_2}{2}\cdot((x')^2 + l\cdot cos(\theta)\cdot x'\cdot\theta' + \frac{l^2}{4}\cdot(\theta')^2 + \frac{m_2}{24}\cdot l^2\cdot(\theta')^2)$$

Potential energy V, taken at the pendulum's centre of mass:

$$V = g\cdot l\cdot cos(\theta)\cdot(m_1 + \frac{m_2}{2})$$

Lagrangian, and the Euler–Lagrange equations with respect to x and θ (F(t) and 0 are the corresponding generalized forces):

$$L = T - V \quad (1)$$

$$F(t) = \frac{d}{dt}(\frac{\partial L}{\partial x'}) - \frac{\partial L}{\partial x} \qquad 0 = \frac{d}{dt}(\frac{\partial L}{\partial \theta'}) - \frac{\partial L}{\partial \theta}$$

Carrying out the derivatives gives the nonlinear equations of motion, Equations (2) and (3):

$$F(t) = (M + m_1 + m_2)\cdot x'' + l\cdot(m_1 + \tfrac{m_2}{2})\cdot cos(\theta)\cdot\theta'' - l\cdot(m_1 + \tfrac{m_2}{2})\cdot sin(\theta)\cdot(\theta')^2 \quad (2)$$

$$0 = l\cdot(m_1 + \tfrac{m_2}{2})\cdot cos(\theta)\cdot x'' + l^2\cdot(m_1 + \tfrac{m_2}{3})\cdot\theta'' - g\cdot l\cdot(m_1 + \tfrac{m_2}{2})\cdot sin(\theta) \quad (3)$$

F(t) is the actuator force on the cart (Equation 2). The pendulum has no external moment applied to it directly, only the indirect coupling through x″ (Equation 3). Both feed the Python simulation (Section 4.1) and the linearised model below.

2.3 Linearization

About the upright equilibrium (θ = 0, θ′ = 0), apply the small-angle approximation (valid up to roughly 10-15°):

$$sin(\theta) \approx \theta \qquad cos(\theta) \approx 1 \qquad \theta\cdot\theta' \approx 0 \qquad (\theta')^2 \approx 0$$

Substituting into Equations (2)-(3) and solving for x″ and θ″ gives Equations (4)-(5):

$$x'' = (\tfrac{C}{\Delta})\cdot F(t) - (\tfrac{B^2 g}{\Delta})\cdot\theta \quad (4)$$

$$\theta'' = -(\tfrac{B}{\Delta})\cdot F(t) + (\tfrac{ABg}{\Delta})\cdot\theta \quad (5)$$

with

$$A = M + m_1 + m_2 \qquad B = l\cdot(m_1 + \tfrac{m_2}{2}) \qquad C = l^2\cdot(m_1 + \tfrac{m_2}{3}) \qquad \Delta = A\cdot C - B^2$$

2.4 State-space Model

With **X = [x, x′, θ, θ′]**^{T} and input **u = F(t)**, Equations (4)-(5) assemble into the standard state-space form, Equation (6):

$$X' = A_{mat}\cdot X + B_{mat}\cdot u,\ y = C_{mat}\cdot X + D_{mat}\cdot u \quad (6)$$

$$A_{mat} = [[0,\ 1,\ 0,\ 0],\ [0,\ 0,\ -\tfrac{B^2 g}{\Delta},\ 0],\ [0,\ 0,\ 0,\ 1],\ [0,\ 0,\ \tfrac{ABg}{\Delta},\ 0]]$$

$$B_{mat} = [0,\ \tfrac{C}{\Delta},\ 0,\ -\tfrac{B}{\Delta}]^T$$

$$C_{mat} = [[1,\ 0,\ 0,\ 0],\ [0,\ 0,\ 1,\ 0]] \qquad D_{mat} = 0$$

y = [x, θ]^T is what the controller actually has: x dead-reckoned from the stepper step count, and θ from the P3022 sensor; neither velocity is measured directly. Numerically (Table 4, Section 8), the open-loop poles are {0, 0, +7.19, –7.19} rad/s which were calculated by deriving the eigenvalues of A_{mat} through linear_model.py (section 4.1). The right-half-plane pole means the plant is unstable without the PD controller of Section 3.

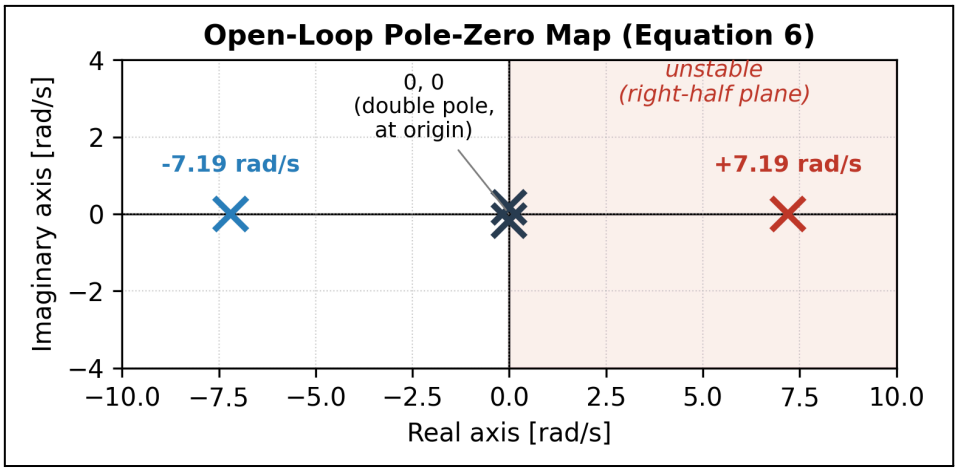


Figure 1: Open-loop pole-zero map

3 Control System Design

3.1 Overview

Two PD loops run on a single actuator command where in both simulation and real hardware, its output is force (Newtons). However, in hardware, the value is then translated to velocity through integration of acceleration ($F = ma$) and step value at 200 Hz. The angle is sensed using a P3022 rotary sensor. It is then driven by the PD controller to bring θ to the upright position at 0°. This action is prioritized over position centering. As for position, it is dead-reckoned from the stepper step count and the position PD gently pulls x towards the center point 0 so the cart stays centred on the rail. The inclusion of a second PD controller for position was necessary due to the limiting usable cart travel length.

3.2 Closed-Loop Architecture

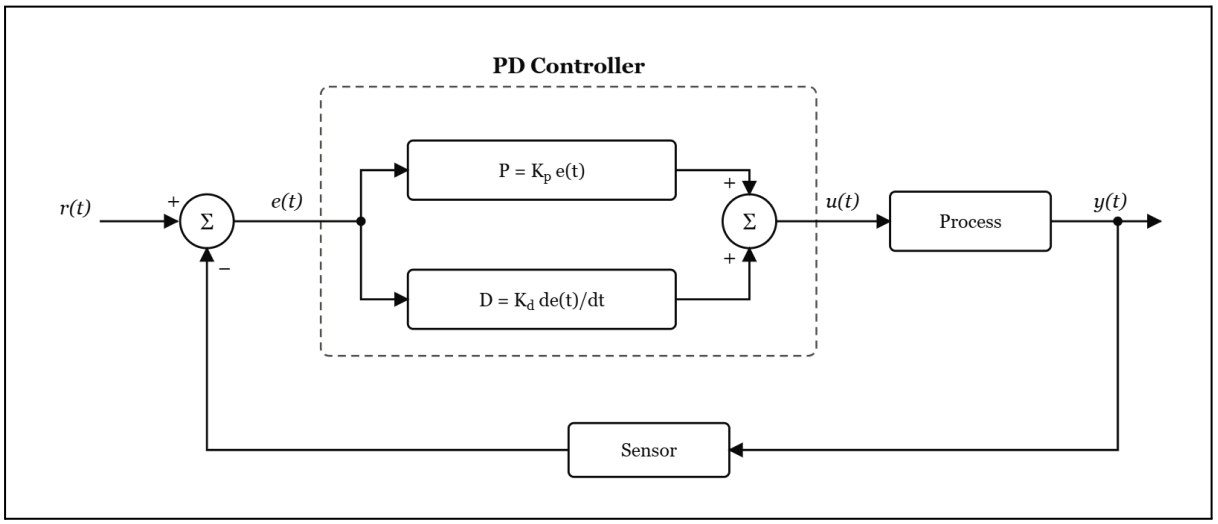


Figure 2: Block Diagram of Closed-Loop Feedback System with PD Control

3.3 PD Control Law

$$u(t) = (K_{p\_\theta} e(t) + K_{d\_\theta} \frac{de(t)}{dt}) + (K_{p\_x} e(t) + K_{d\_x} \frac{de(t)}{dt}) \quad (7)$$

3.4 Gain Tuning Methodology

Treating the pendulum angle dynamic as a damped second-order system, and choosing a target natural frequency ω_n and damping ratio ζ, the linearised θ″ equation (Section 2.3) gives the gains directly, Equations (8)-(11):

$$K_{p\_\theta} = A \cdot g + (\frac{\Delta}{B}) \cdot \omega_n^{\,2} \quad (8)$$

$$K_{d\_\theta} = 2\zeta \cdot (\frac{\Delta}{B}) \cdot \omega_n \quad (9)$$

$$K_{p\_x} = (5\text{–}20\%) \cdot K_{p\_\theta} \quad (10)$$

$$K_{d\_x} = 2\zeta \cdot \sqrt{(K_{p\_x} \cdot M_{total})} \quad (11)$$

The position loop is kept much weaker than the angle loop so re-centring the cart never competes with staying upright (Section 3.1). These estimates were refined in simulation by sweeping initial θ = −15°…+15° (Section 4.1), giving the preset gains K_{p_θ} = 9.17, K_{d_θ} = 0.75, K_{p_x} = 1, K_{d_x} = 1.5. The hardware was then re-tuned from small values, since the simulation actuator is a force and the stepper is a velocity command (Section 4.2).

The derivative term in both loops is also low-pass filtered (pd_controller.py, arduino_IP.ino) so sensor noise is not amplified by differentiation. The filter's time constant τd sets its cutoff frequency, Equation (12):

$$f_{\tau d\_cutoff} = 1 / (2\pi \cdot \tau d) \quad (12)$$

With τd = 0.02 s, $f_{\tau d\_cutoff}$ ≈ 8 Hz, comfortably above the pendulum's own ≈1.1 Hz dynamics (the ±7.19 rad/s eigenvalues of Section 2.4) so the filter adds little phase lag to the signal the derivative term needs to track, while still smoothing sample-to-sample ADC noise well below the 200 Hz control rate.

4 Software Implementation

4.1 Simulation - Python

The Python simulation (Python_Simulation/) implements the nonlinear plant (Equations 2-3) and the PD controller (Equation 7), so the control law could be designed and checked in software before it went on the physical rig.

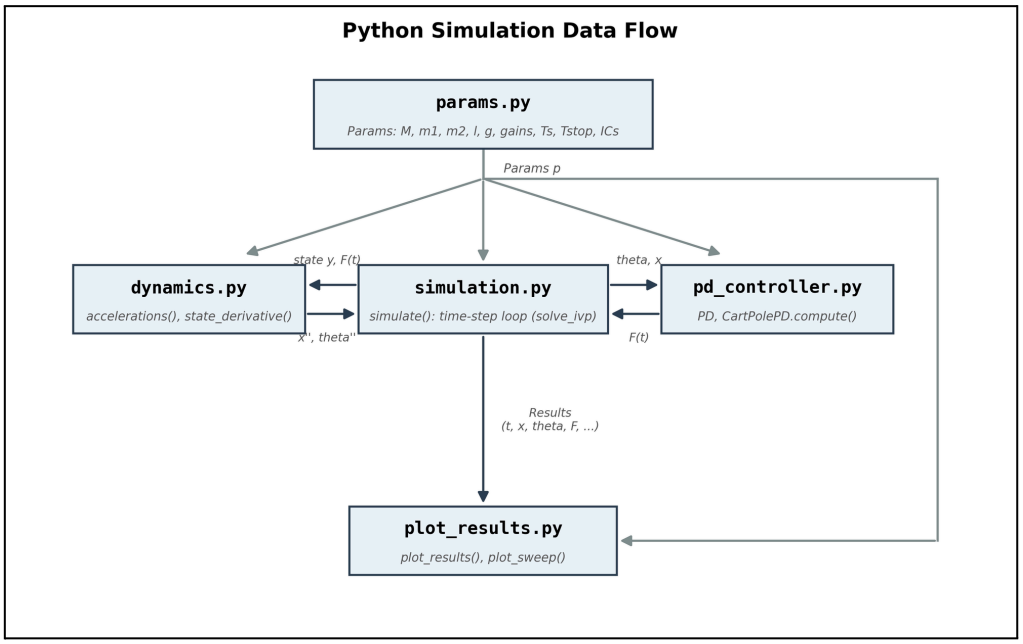


Figure 3: Simulation data flow between params.py, dynamics.py, pd_controller.py, simulation.py, and plot_results.py

dynamics.py solves the 2×2 mass matrix for x″ and θ″ at every step (np.linalg.solve), and **simulation.py** integrates the result with SciPy's solve_ivp ("RK45") at Ts = 5 ms, the same rate used on the hardware. **linear_model.py** builds the state-space matrices of Equation (6) and computes the open-loop poles as the eigenvalues of A_{mat} (open_loop_poles()), where main.py prints these at start-up to confirm the instability before any control is applied. The CartPolePD class (**pd_controller.py**) sums two PD loops, angle and position, into one cart force, saturated at ±60 N with a filtered derivative and conditional anti-windup. Only kp and kd are tuned for either loop (Section 3.4), so the law actually exercised in this report is the PD law of Equation (1), not a tuned PD. **main.py** runs a single balance (python main.py) or a full angle sweep (python main.py --sweep, Figure 9, Section 6.1). --gains preset uses the tuned gains of Section 3.4, and --gains spec uses the intentionally weak defaults that fall over, to show why tuning mattered.

4.2 Embedded Controller - ARDUINO

The embedded controller (Arduino_Control/arduino_IP/arduino_IP.ino) runs the same PD law on the physical rig at a fixed 200 Hz, matching the simulation's step time (Ts).

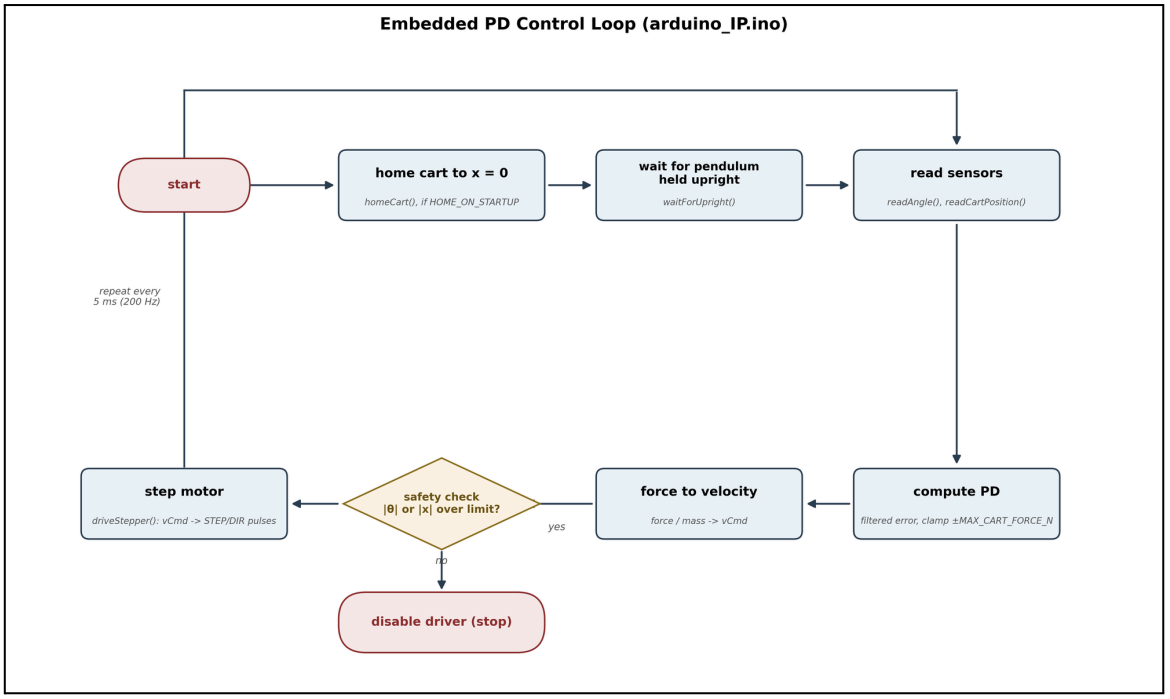


Figure 4: Flowchart of embedded control loop

Each tick it reads the P3022 sensor (readAngle(), 6× oversampled) and the dead-reckoned cart position (readCartPosition()), then computes a filtered derivative error using the derivative cutoff frequency (Equation 12) for both loops (computePD()) and sums them into a force command clamped to ±MAX_CART_FORCE_N. Due to the fact that the stepper is a velocity command, not a force, that force is integrated (force/mass) into a commanded velocity vCmd and converted to STEP/DIR pulses (driveStepper()). This is the one place the simulation and the hardware genuinely differ. A safety check (safetyCheck()) disables the driver if |θ| or |x| exceed their limits. At start-up the sketch can crash-home the cart to x = 0 (HOME_ON_STARTUP) and waits for the pendulum to be held upright before arming the loop (waitForUpright()). The hardware gains (K_{p_θ} = 11.0, K_{d_θ} = 1.7, K_{p_x} = 2.0, K_{d_x} = 1.2) were re-tuned on the bench from small values, using the simulation's preset gains only as a reference (Section 3.4).

5 Hardware Implementation

5.1 Mechanical Design

The rig was designed in Fusion 360 as two STEP assemblies (CAD/Assemblies/, custom parts in CAD/Custom/, vendor parts in CAD/Vendor/), shown in Figure 5. The cart rides a 0.540 m T-slot rail (usable half-travel ≈ 0.194-0.214 m) driven by a GT2 belt/pulley, while the pendulum pivots in a single vertical plane.

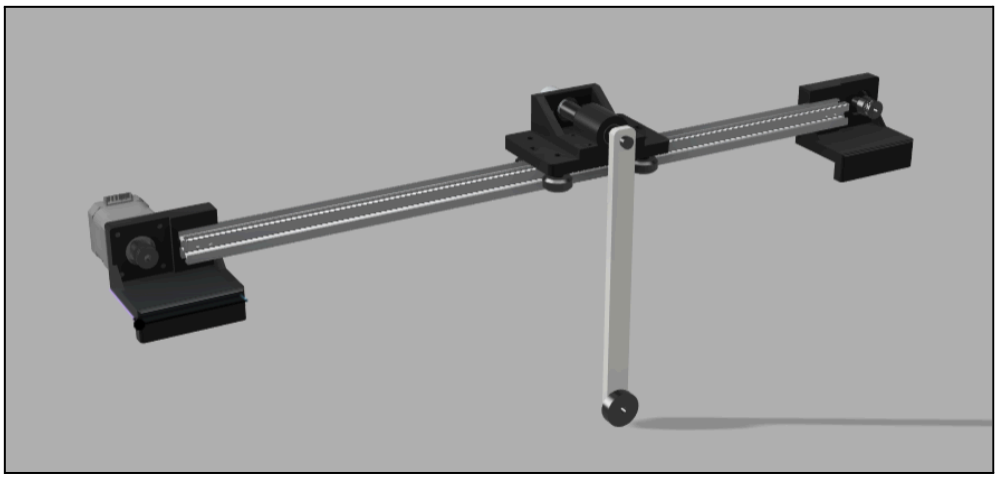


Figure 5: Fusion 360 master-assembly render

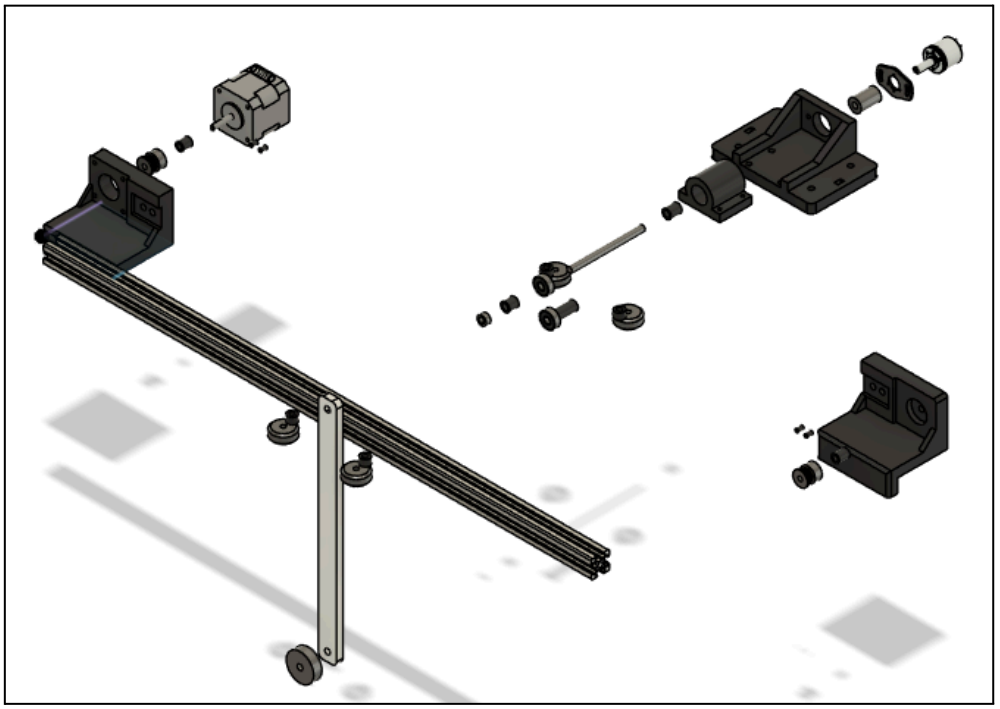


Figure 6: Exploded-view of master assembly

Custom parts (platform, bearing slider/housing, pendulum stick, T-slot holders, spacers, drill guides) were 3D-printed in PLA. Bearings, shaft, motor, and fasteners are off the shelf. Printed drill guides (STL(3d_printing)/Drill_Guide/) kept hole placement repeatable. Full BOM in Table 1.

Component	Spec / Notes	Qty	Source
Arduino Uno R3 Kit (includes: capacitors, resistors, 26-28 AWG jumper wires, breadboard)	ATmega328P, USB-powered logic, Prototype wiring, Noise filtering	1 set	Amazon
DRV8825 stepper driver	STEP/DIR driver	1	Amazon
NEMA 17 stepper motor	1.8°/step (200 steps/rev), 4-wire bipolar	1	Amazon
P3022 rotary angle sensor	12-bit (read 10-bit on Arduino A0)	1	Amazon
12–24 V DC power supply	Motor power only	1	Amazon
GT2 belt + 20T pulley	2 mm pitch, 0.040 m circumference	1 set	Amazon
Aluminium T-slot rail	0.540 m linear axis	1	Amazon
606-2RS bearings	Pendulum pivot bearings	2	Amazon
V-slot 625ZZ bearings	Cart roller bearings	4	Amazon
D-shaft + shaft coupler + collar	Pendulum pivot	1 set	Amazon
Pendulum rod + end mass	m2 = rod (l = 0.250 m), m1 = end bob	1	Printed
3D-printed PLA parts	Platform, slider, housings, holders, spacers, drill guides	set	Printed
M3 / M5 / M6 fasteners + spacers	Assembly hardware	as needed	Amazon / Printed
22 AWG copper stranded wire	Motor power transmission wires	1 set	Amazon

Table 1: Bill of materials

5.2 Electronics

Wiring is captured in a KiCad project (IP_Circuit_Schematics/): Arduino Uno R3, DRV8825 driver, P3022 sensor, 220 ohm resistor & 10µF capacitor for P3022 output noise filtering, and a 100 µF smoothing capacitor across the driver's motor-supply input as shown in Figure 7 below.

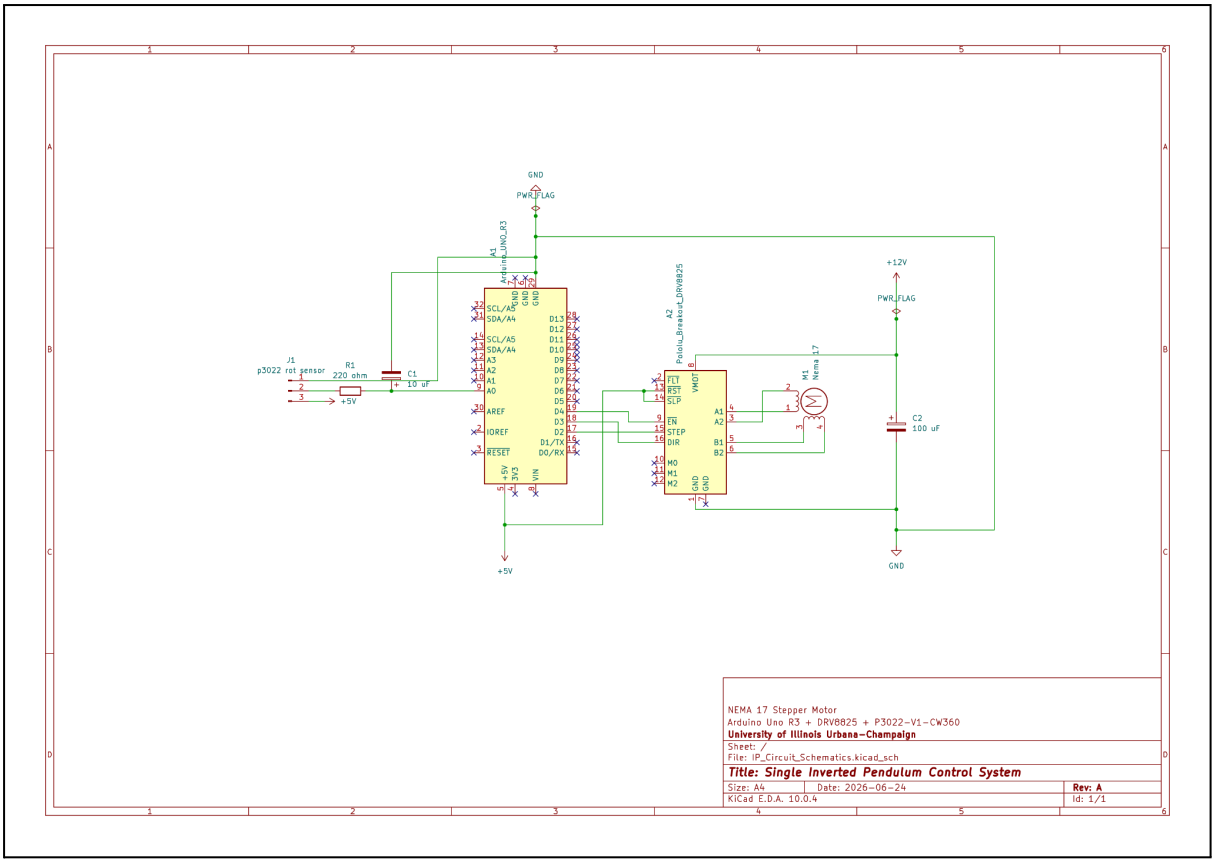


Figure 7: KiCad circuit schematic

The 10 µF capacitor was selected because it was the smallest capacitance available in my inventory. A 220 Ω resistor was then chosen to pair with the capacitor by applying the Nyquist sampling criterion and the RC cutoff frequency equation. Equations (13)-(14) as shown below.

$$f_{Nyquist} = \frac{f_{sampling}}{2} = \frac{200Hz}{2} = 100Hz \tag{13}$$

$$f_{RC\_cutoff} = \frac{1}{2\pi \cdot R \cdot C} = 72.3Hz \tag{14}$$

A 100 µF smoothing capacitor was placed across the DRV8825 motor-supply input following the manufacturer's recommendation to provide bulk energy storage, reduce supply voltage fluctuations, and protect the driver from inductive voltage spikes generated during stepper motor operation

Signal	Arduino / Driver Pin	Notes
P3022 OUT	A0	angle = SENSOR_SIGN × (raw − ANGLE_OFFSET_COUNTS) × (360/1024)
P3022 VCC / GND	5 V / GND	shared ground with driver and motor supply
DRV8825 STEP	D2	one microstep per pulse
DRV8825 DIR	D3	sets direction
DRV8825 EN	D4 (active-low)	disabled on a safety trip

Motor power	separate 12–24 V DC	never from the Arduino 5 V rail

Table 2: Pin connections between the P3022 sensor, the DRV8825 driver, and the Arduino Uno

The Arduino runs on USB 5 V logic only while the NEMA 17 draws from a separate 12-24 V supply through the driver. With 200 full steps/rev and the 0.040 m pulley, METERS_PER_STEP = 0.040/(200 × microsteps). The NEMA 17's two coil pairs (found by continuity check) go to the driver's A+/A− and B+/B− terminals.

5.3 Hardware Validation Protocols

The hardware was brought up in stages, each verified before it was trusted inside the closed loop, shown in Figure 8.

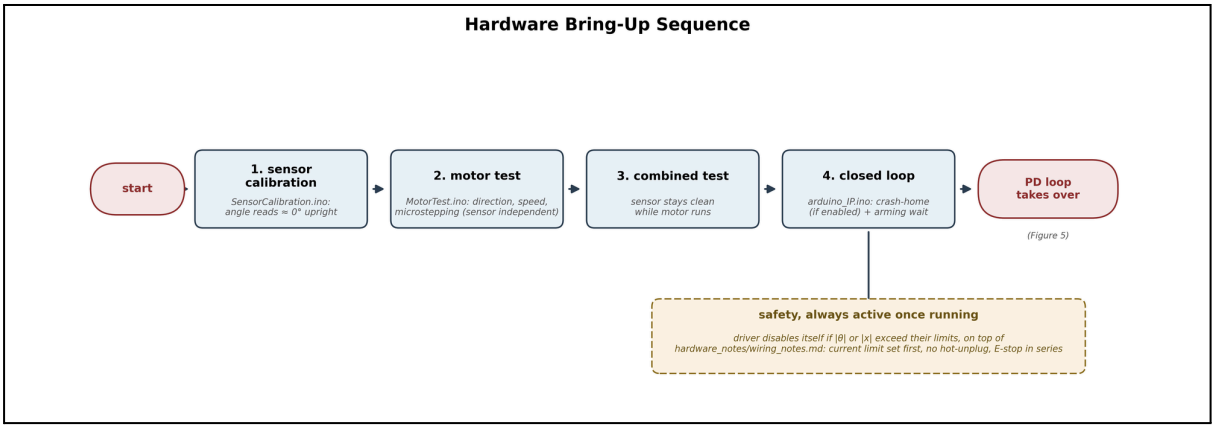


Figure 8: Flowchart of hardware bring-up sequence

Sensor calibration confirms the angle reads ≈ 0° upright and motor testing confirms direction, speed, and microstepping independent of the sensor. The reason is because a combined test checks if the sensor stays clean while the motor runs. Only then is the full arduino_IP.ino sketch run as a closed loop, with crash-homing (if enabled) and an arming wait before the PD loop takes over. The driver disables itself if |θ| or |x| exceed their limits: set the driver current limit first, never hot-unplug the motor, and add a physical E-stop in series with the motor supply.

6 Results

6.1 Simulation Environment

With the preset PD gains (K_{p_θ} = 9.17, K_{d_θ} = 0.75, K_{p_x} = 1, K_{d_x} = 1.5), every starting angle from −15° to +15° recovers to upright with no rail contact, and re-centres within roughly 3-4 s (Table 3, Figure 9). The opposite-side overshoot is 60-65% of θ_0, and the cart excursion stays well inside the ±21.4 cm rail half-travel.

θ_0 (deg)	Peak |θ| (deg)	Opposite-side overshoot (deg)	Peak |x| (cm)	Result
±15	15.00	10.0	16.0	Upright
±10	10.00	6.5	10.4	Upright
±5	5.00	3.2	5.1	Upright
0	0.00	0.0	0.0	Upright

Table 3: Simulated peak angle, overshoot, and cart excursion vs. starting angle (preset PD gains)

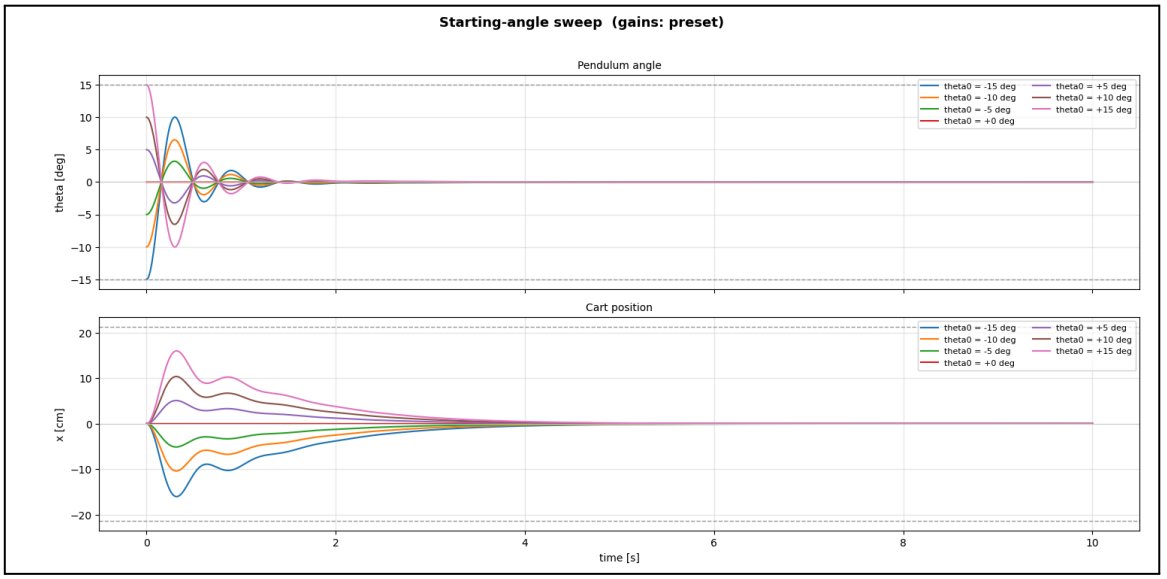


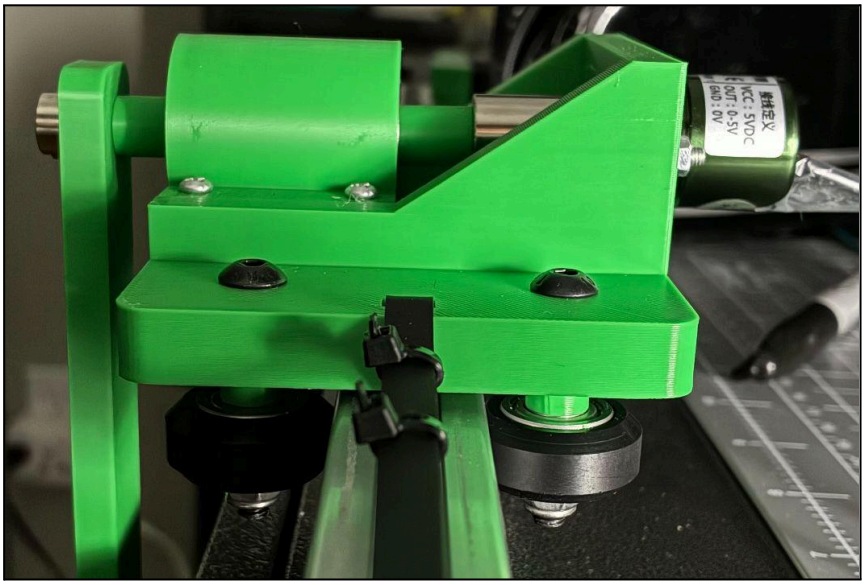
Figure 9: Simulated pendulum angle (top) and cart position (bottom) vs. time for θ_0 = −15°…+15° (preset PD gains, python main.py --sweep)

6.2 Hardware System

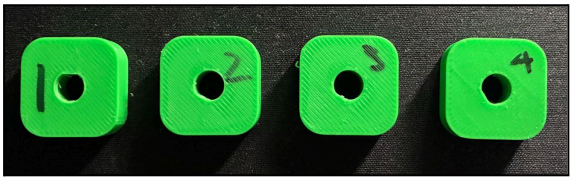
As already reported in Section 1.2, the hardware settles to within ±0.5° in ≈0.7 s and re-centres to within ±1 cm in ≈1.4 s, with overshoot and cart excursion in the same range as simulation (60-65% of θ_0, 2.5-4.7 cm). The demonstrated recoverable range (±7°) is narrower than simulation's (±15°), most likely from the 10-bit ADC resolution, the discretised velocity-command stepper versus the simulation's continuous force, and unmodeled belt/bearing friction. Cart force stayed within ±60 N throughout.

6.3 Mechanical Challenges

Most of the mechanical difficulty was in alignment and repeatability, not the parts themselves (Figure 10). Printed drill guides (STL(3d_printing)/Drill_Guide/) kept rail and platform hole placement accurate enough for the cart to roll freely, and several print iterations were needed to get PLA part tolerances around the bearings and D-shaft right without forcing. Moreover, because the cart position is dead-reckoned from the step count, any belt slack or missed step corrupts the position estimate directly, so belt/pulley fit mattered as much as gain tuning. Finally, the crash-homing routine exists to re-establish a known x = 0 at start-up rather than assume it.



Cart bearings and T-slot fit



Pendulum stick D-shaft hole
(Tight Fit [Left] -> Loose Fit [Right])

Drill guide
(Helps draw precise drilling point)

Figure 10: A mechanical fit/alignment issue encountered during assembly.

7 Conclusion and Future Work

7.1 Lessons Learned

The biggest lesson was control system design: stabilising an open-loop-unstable plant (a pole at +7.19 rad/s), estimating PD gains from a damped second-order heuristic model instead of pure trial and error, and balancing two competing objectives, upright angle and a centred cart, through one actuator command. Deriving the nonlinear equations of motion with Lagrange's method and building the A/B/C/Δ model let stability be checked directly from eigenvalues rather than by trial and error on hardware. On the embedded and mechanical side, writing a fixed-rate (Ts = 5 ms) real-time loop in C++/Arduino, and designing a belt-driven cart on a T-slot rail with 3D-printed parts, both required planning for travel limits and manufacturability up front (Section 6.3). None of it worked until the staged bring-up of Section 5.3, calibration, then motor test, then a combined test, and only then the closed loop, was in place.

7.2 Future Improvements

Control theory: replace the hand-tuned PD law with LQR on the state-space model of Section 2.4, and add a better filtering system like Kalman filter to estimate velocity instead of differentiating noisy measurements.

Hardware: read the P3022 digitally for its full 12-bit resolution, add a real cart-position encoder or limit switches instead of dead-reckoning, improve on noise filtering measurements by utilizing a ceramic capacitor with lower capacitance, and implement a larger gear ratio for faster response and larger recoverable angle.

8 Appendix

Detailed parameters, derived constants, and analysis, matching the current Python_Simulation/params.py and Arduino_Control/arduino_IP/arduino_IP.ino. Table 3 lists the physical parameters used throughout this report.

Physical parameters

Symbol	Meaning	Value
M	Cart mass	0.220 kg
m_1	End bob (point mass at rod tip)	0.042 kg
m_2	Pendulum rod mass	0.018 kg
l	Rod length	0.250 m
g	Gravity	9.81 m/s²
Rail length / cart length	Usable half-travel = (0.540 – 0.112)/2	0.540 m / 0.112 m → 0.214 m

Table 4: Physical parameters of the cart-pendulum system

Derived constants and stability: evaluating A, B, C, and Δ from Section 2.3 with the physical parameters of Table 4 gives

$$A = M + m_1 + m_2 = 0.280\ kg$$

$$B = l\cdot(m_1 + \tfrac{m_2}{2}) = 0.250\cdot(0.042 + 0.009) = 0.01275$$

$$C = l^2\cdot(m_1 + \tfrac{m_2}{3}) = 0.0625\cdot(0.042 + 0.006) = 0.00300$$

$$\Delta = A\cdot C - B^2 = 0.00084 - 0.0001626 = 0.000677$$

so the open-loop (plant) poles are {0, 0, +7.19, −7.19} rad/s. The single right-half-plane pole confirms the plant is unstable, and the minimum K_{p_θ} needed just to balance is A·g = 0.280 × 9.81 = 2.75 N/rad.

Numeric state-space matrices: substituting the same constants into Equation (6) gives

$$A_{mat} = [[0,\ 1,\ 0,\ 0],\ [0,\ 0,-\ 0.240,\ 0],\ [0,\ 0,\ 0,\ 1],\ [0,\ 0,\ 51.66,\ 0]]$$

$$(row\ 2 = -\tfrac{B^2 g}{2},\ row\ 4 = \tfrac{ABg}{\Delta})$$

$$B_{mat} = [0,\ 4.430,\ 0,-\ 18.83]^{\mathrm{T}}\ \ (=\ [0,\ C/\Delta,\ 0,\ \ -\ B/\Delta]^{\mathrm{T}})$$

$$C_{mat} = [[1,\ 0,\ 0,\ 0],\ [0,\ 0,\ 1,\ 0]] \qquad D_{mat} = 0$$

Controller gains: Table 5 compares the simulation preset gains against the gains currently in arduino_IP.ino.

Gain	Simulation (preset)	Hardware (arduino_IP.ino)
K_{p_θ}	9.17	11.0
K_{d_θ}	0.75	1.7
K_{p_x} / K_{d_x}	1 / 1.5	2.0 / 1.2
Derivative filter τd	0.02 s	0.02 s
Actuator limit	±60 N (force)	±60 N (force) → step interval 500–6100 µs

Table 5: Controller gains, simulation preset vs. hardware

Hardware constants: the following geometric and safety constants are taken directly from arduino_IP.ino as currently checked in:

$$METERS\_PER\_STEP = 0.040 / (200 \times 1) = 2.0{\times}10^{-4}\, m/step$$

(0.20 mm/step, full-step; MICROSTEPS = 1 in the current sketch)

$$control\ rate = 5\, ms = 200\, Hz$$

$$ADC\ scale = 360 / 1024 = 0.3516\, deg/count\ (10\, bit)$$

$$safety\ trips = |\theta| > 90°\ (7\ sample\ debounce)\ \ or\ \ |x| > 0.22\, m$$

Note that these safety-trip and step-geometry constants reflect arduino_IP.ino as currently checked in. Earlier design notes used a tighter |θ| > 15° / |x| > 0.194 m trip and a 16× microstep setting.

9 Project Repository

- **GitHub repository:** github.com/endotakeru/Single_Inverted_Pendulum/tree/main
- **CAD files:** /CAD (STEP assemblies in Assemblies/, custom parts in Custom/, vendor parts in Vendor/) and /STL(3d_printing) (printable parts, including Drill_Guide/).
- **Hand calculations:** drive.google.com/file/d/140igMz2Y46unPIWgm3RWE6VgtJxZAJj0/view?usp=sharing
- **Schematics:** /IP_Circuit_Schematics (IP_Circuit_Schematics.kicad_sch).
- **Simulation code:** /Python_Simulation (main.py, dynamics.py, linear_model.py, pd_controller.py, simulation.py, plot_results.py, params.py).
- **Embedded code:** /Arduino_Control/arduino_IP/arduino_IP.ino, plus any calibration or motor-test sketches used during bring-up (Section 5.3).
- **BOM:** Table 1 (Section 5.1).
- **Interactive demo:** /Dashboard_Simulation/index.html, a browser version of the simulator.